



PROJECT REPORT: CINESCOPE

Name: Safwan Salehin

Roll: 2207044

Course NO: CSE-3110

Course Title: Database Systems Laboratory

Group: A2

Date: July 19th, 2026

SUBMITTED TO:

Most. Kaniz Fatema Isha

Lecturer

Computer Science and Engineering
Khulna University of Engineering & Technology

Waliul Islam Sumon

Lecturer

Computer Science and Engineering
Khulna University of Engineering & Technology

INTRODUCTION

CineScope is built with the goal to display various features of ORACLE database combined/merged into one web application. It is a database-driven application designed to demonstrate the integration of Oracle PL/SQL features. The project provides a structured platform to manage movies, directors, production companies, and user reviews while showcasing advanced database concepts such as triggers, views etc. Key implementations include triggers for automatic rating updates, procedures and functions, constraints and aggregate functions for data integrity, and views for simplified data access. Advanced PL/SQL constructs such as cursors, user-defined types, dynamic arrays, and SYSDATE-based date calculations are employed to enrich functionality. On the application side, Blade templates and controllers enable user-friendly forms and dynamic homepage sections, including recent reviews and anniversaries. This project demonstrates practical skills in combining relational database design with web application development, resulting in a robust and visually engaging system.

Project Objectives

- 1) A visual webpage/web application to display ORACLE database activity.
- 2) Various database features to be shown through various actions from the application from user/guest/admin side.
- 3) User role demonstration to provide different levels of interaction with the database.
- 4) Display and visualize advanced ORACLE features.

Technology Stack

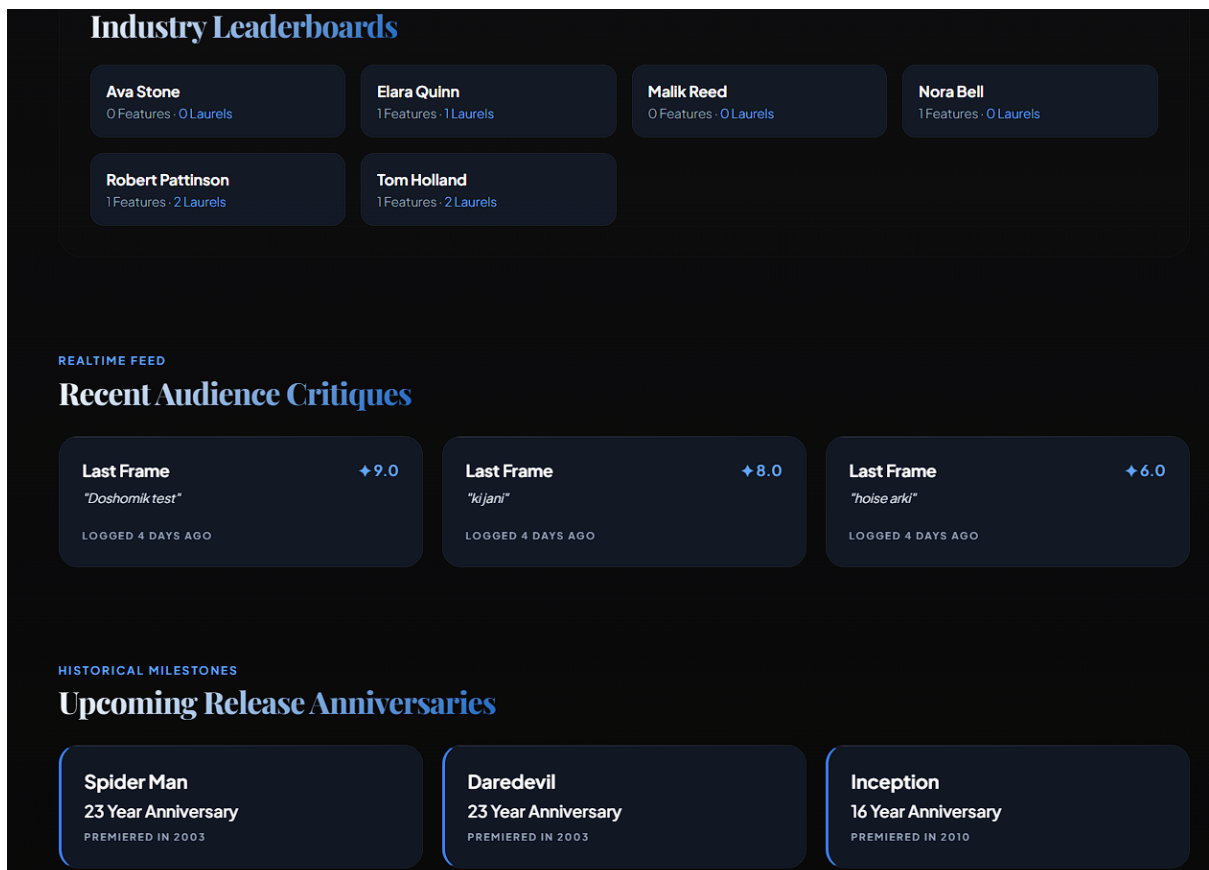
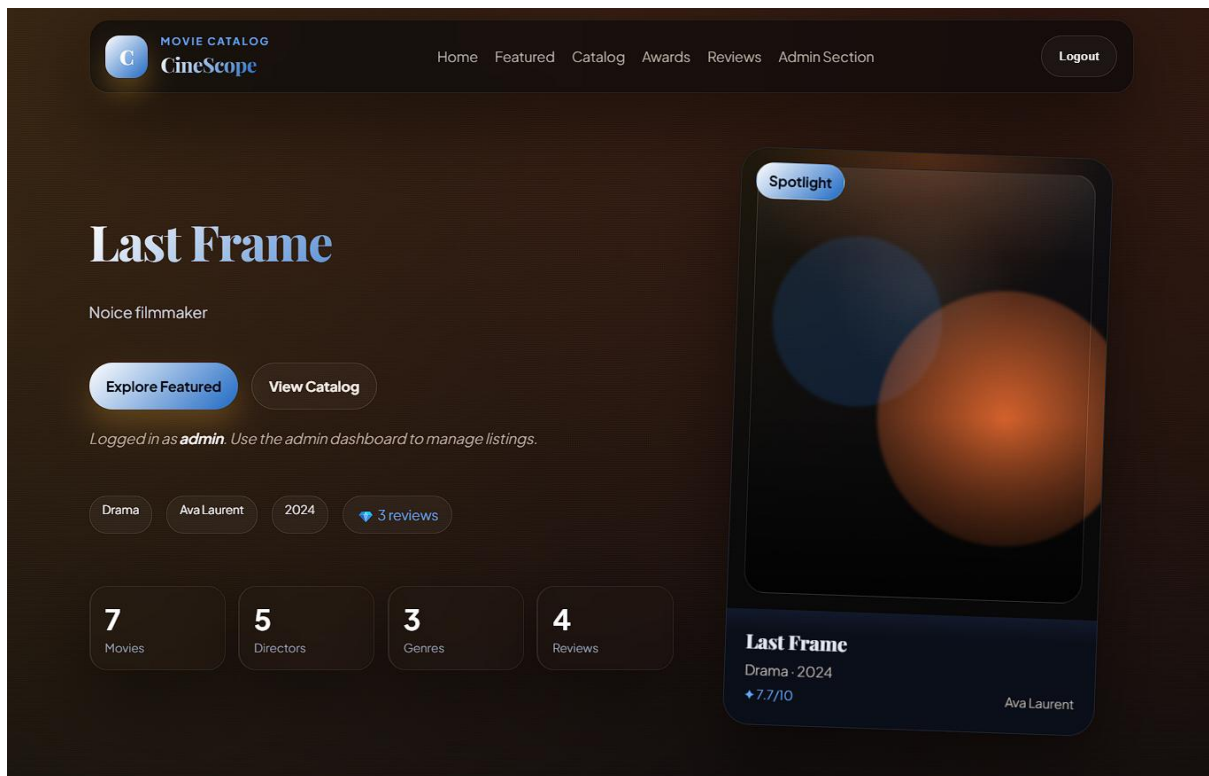
- Database Engine: ORACLE Database.
- Backend Server Application: PHP(Laravel Framework).
- Frontend UI Presentation: HTML5, CSS and JavaScript.

Website Features

1. Homepage

- What it shows:
 - Top navigation bar to view each part and login/logout. Also some specific features depending on whether the user is normal user or admin.
 - Some database stats and featured movie descriptions.
 - Search Engine for specific movie details such as actors, ratings, reviews, production companies etc.
 - Achievement for various persons i.e number of movies they made or awards they got.
 - A section for "Recent Reviews" (pulled from the recent_reviews view).

- A section for “Upcoming Movie Anniversaries” (pulled from the movie_anniversaries view).
- User role: Guest can browse without logging in. Admin and users can log in for their specific feature advantages.



2. Movies Index Page (For Admin)

- What it shows:
 - A list of all movies with title, release year, rating, language, director, and production company.
 - Buttons: Edit, Delete, Add New Movie.

3. Add New Movie Page (For Admin)

- Form fields: Title, Release Year, Rating, Language, Director (dropdown or add new), Production Company (dropdown), Poster URL, Description.
- Buttons: **Save Movie**, **Cancel**.

4. Edit Movie Page (For Admin)

- What it shows:
 - Pre-filled form with existing movie details.
 - Dropdowns for director and production company.
- Buttons: Update Movie, Cancel.

5. Delete Movie (For Admin)

- What it does:
 - Removes a movie record.
 - Cascade deletes linked reviews and production company relations.

6. Reviews Section

- What it shows:
 - Users can add text reviews + ratings(0-10).
 - Recent reviews of this user displayed with name, comment, rating.
- User role: Logged-in users can add reviews; guests can only read.

7. User Authentication

- What it does:
 - Login and signup forms.
 - Admin dashboard accessible only after login with admin credentials.
- User role:
 - Guest: Browse only.
 - User: Add reviews.
 - Admin: Full CRUD on movies, directors etc.

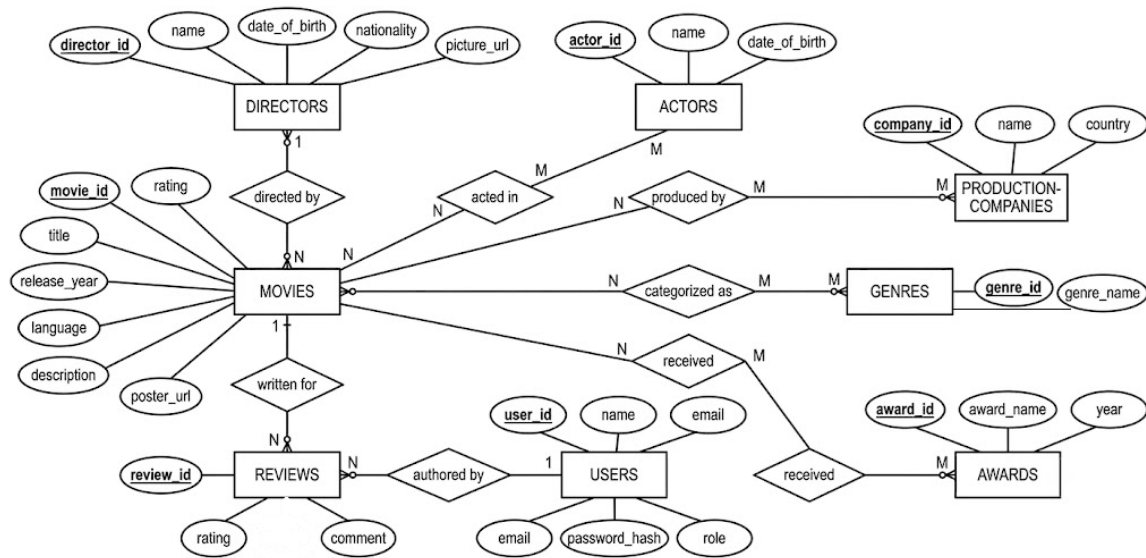
8. Admin Dashboard

- What it shows:
 - Same as guests/users.
 - Extra navigation button for admin section operations.
 -

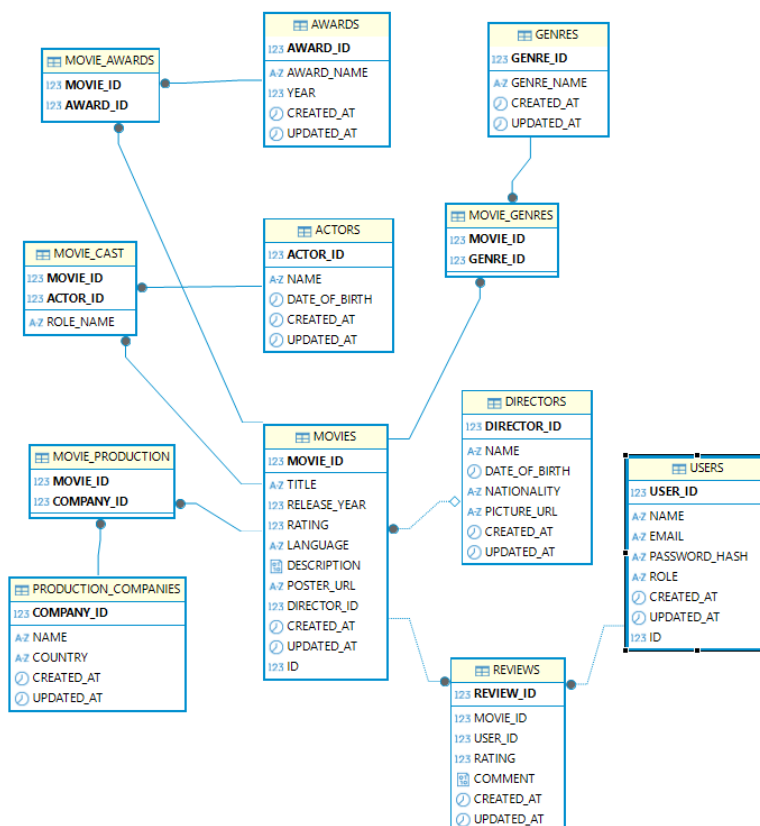
System Architecture

DATABASE DESIGN

ER Diagram



Schema Diagram



Data Dictionary

Movies

- **Purpose:** Stores core information about each movie in the catalog.
- **Columns:**
 - movie_id (PK, NUMBER) – Unique identifier for each movie.
 - title (VARCHAR2(255)) – Movie title.
 - release_year (NUMBER) – Year of release.
 - rating (NUMBER(3,1)) – Average rating (0–10).
 - language (VARCHAR2(50)) – Language of the movie.
 - description (CLOB) – Synopsis or description.
 - poster_url (VARCHAR2(500)) – Link to poster image.
 - director_id (FK → Directors.director_id) – Associated director.

Directors

- **Purpose:** Stores information about movie directors.
- **Columns:**
 - director_id (PK, NUMBER) – Unique identifier.
 - name (VARCHAR2(100)) – Director's name.
 - date_of_birth (DATE) – Birth year/date.
 - nationality (VARCHAR2(100)) – Nationality.
 - picture_url (VARCHAR2(500)) – Optional photo link.

Production_Companies

- **Purpose:** Stores film production companies.
- **Columns:**
 - production_company_id (PK, NUMBER) – Unique identifier.
 - name (VARCHAR2(150)) – Company name.

Movie_Production

- **Purpose:** Junction table linking movies to production companies.
- **Columns:**
 - movie_id (FK → Movies.movie_id) – Movie reference.
 - company_id (FK → Production_Companies.company_id) – Company reference.
- **Constraints:** Composite PK (movie_id, company_id).

Actors

- **Purpose:** Stores actors who appear in movies.
- **Columns:**
 - actor_id (PK, NUMBER) – Unique identifier.
 - name (VARCHAR2(100)) – Actor's name.
 - date_of_birth (DATE) – Actor's date of birth.

Movie_Cast

- **Purpose:** Junction table linking actors to movies with roles.
- **Columns:**
 - movie_id (FK → Movies.movie_id).
 - actor_id (FK → Actors.actor_id).
 - role_name (VARCHAR2(150)) – Character name.
- **Constraints:** Composite PK (movie_id, actor_id).

Awards

- **Purpose:** Stores award information.
- **Columns:**
 - award_id (PK, NUMBER).
 - award_name (VARCHAR2(150)).
 - year (NUMBER).

Movie_Awards

- **Purpose:** Junction table linking movies to awards.
- **Columns:**
 - movie_id (FK → Movies.movie_id).
 - award_id (FK → Awards.award_id).
- **Constraints:** Composite PK (movie_id, award_id).

Reviews

- **Purpose:** Stores user reviews for movies.
- **Columns:**
 - review_id (PK, NUMBER).
 - movie_id (FK → Movies.movie_id).
 - user_id (FK → Users.user_id).
 - comment (CLOB).
 - rating (NUMBER(3,1)).
 - created_at (DATE).
- **Special:** Trigger updates Movies.rating when new review is added.

Users

- **Purpose:** Stores application users (guest, user, admin).
- **Columns:**
 - user_id (PK, NUMBER).
 - name (VARCHAR2(100)).
 - email (VARCHAR2(150), UNIQUE).
 - password_hash (VARCHAR2(255)).
 - role (VARCHAR2(20)) – Values: guest, user, admin.
 - active (BOOLEAN/NUMBER(1)) – Active/inactive status.

IMPLEMENTED ADVANCED DATABASE FEATURES

TRIGGERS

1. Automatic Average Rating Tracker (Database Trigger): When new evaluations are written to the system, an AFTER INSERT trigger automatically calculates the aggregate arithmetic mean (AVG) of all combined rating fields for that specific movie ID. The resulting score is immediately saved back into the parent MOVIES row.
2. Trigger for auto increment of ID: Since using SEQUENCE feature is not allowed, auto increment of ID for every table is done using trigger. Each table requires a single trigger.
3. Trigger for Awards year validation check: A trigger that ensures no movie gets an award before it is released.

Custom PL/SQL Procedures and Functions

1. `add_movie_with_director` (procedure): An operation that records a new movie profile while dynamically confirming or generating a valid director row.
2. `get_upcoming_anniversaries`(procedure): Calculates upcoming anniversary for movies.
3. `years_since_release`(function): A precise calculation function using SYSDATE calculations to compute exact historical milestones for release logs.

User data types

To support bulk procedural array actions inside custom view loops, specific structural data types are declared directly in the Oracle instance:

1. `anniversary_obj`: A composite structural object mapping title strings, calculated ages, and specific performance tracking fields.
2. `anniversary_tab`: A memory-resident collections array type defined as "IS TABLE OF `anniversary_obj`".

Views

1. `recent_reviews`: An optimized system view that returns the top 3 latest review records written across application tables.
2. `movie_anniversaries`: A reporting layer view linking memory arrays and current date attributes using cursors to compute upcoming historic milestones.

CONCLUSION

CineScope is a web application for movie associated informations built using Laravel Framework and ORACLE SQL database. The application delivers views like a real life IMDB website for movies and keeps things interactive for users such as review options for users. With the views and other interactive options, database features have been clearly demonstrated. Every database action works in the background and the result is displayed in the website views. Various advanced features such as triggers, functions, PL/SQL, user-defined data types, dynamic arrays are also used in specific website features. The website provides a clear and engaging way to interact with the database, offering features like movie catalog browsing, director and production company management, reviews with automated rating updates, award tracking, actor leaderboards, and upcoming anniversaries. Role-based access ensures that guests, users, and administrators experience tailored functionality, reinforcing both usability and security. Ultimately, the system stands as a comprehensive showcase of how database activity can be effectively visualized and managed through a web application.